

SNIES MONITOR · UNINORTE

Manual completo del proyecto

Cómo funciona el scraping, la clasificación de novedades,
el correo automático, GitHub Actions y el dashboard

Generado en julio de 2026 a partir de una revisión completa del código fuente

Manual completo — SNIES Monitor · Uninorte

Este documento explica, en lenguaje sencillo, cómo funciona **todo** el proyecto: desde que un robot entra a la página del gobierno a descargar un Excel, hasta que llega un correo y se actualiza la página web. Está pensado para alguien con conocimientos básicos de programación, así que cada vez que aparece un término técnico se explica la primera vez que se usa.

Si en algún momento solo necesitas hacer algo puntual (cambiar el correo, cambiar destinatarios, entender un gráfico), puedes ir directo a esa sección con el índice. No hace falta leer todo de corrido.

Índice

1. ¿Qué es este proyecto y para qué sirve?
 2. El flujo completo, de principio a fin
 3. Mapa de carpetas y archivos
 4. Paso 1 — Cómo se descarga la información (web scraping)
 5. Paso 2 — Qué información se guarda de cada programa
 6. Paso 3 — Cómo se decide si un programa es Nuevo, Inactivo o Modificado
 7. Paso 4 — Cómo se guarda el historial (acumulación en Excel)
 8. Paso 5 — El correo automático
 9. Paso 6 — El Dashboard (la página web)
 10. GitHub Actions: el robot que hace todo esto solo
 11. Los "Secrets" de GitHub
 12. Cómo cambiar los destinatarios del correo
 13. Cómo cambiar la cuenta de correo que envía los reportes
 14. GitHub Pages: cómo se publica la página web
 15. Cómo correr todo esto en tu propio computador
 16. Problemas comunes y cómo resolverlos
 17. Cosas importantes que hay que saber (y detalles desactualizados)
 18. Glosario de términos técnicos
-

1. ¿Qué es este proyecto y para qué sirve?

El **SNIES** (Sistema Nacional de Información de la Educación Superior) es una base de datos pública del Ministerio de Educación de Colombia donde **todas** las universidades del país deben registrar sus programas académicos (pregrado, posgrado, etc.).

Cualquiera puede consultarla en:

<https://hecaa.mineducacion.gov.co/consultaspublicas/programas>

El problema es que esa página no avisa cuando algo cambia. Si la Universidad Javeriana abre una carrera nueva, o Uniminuto le baja los créditos a un programa, o una institución da de baja una carrera, **nadie te lo notifica** — tendrías que entrar manualmente todos los días y comparar a ojo entre miles de programas.

Este proyecto (`snies-monitor`) automatiza justamente eso, para que la Dirección de Planeación de Uninorte pueda vigilar a la competencia sin trabajo manual:

1. Cada semana, un robot entra a la página del SNIES y descarga el Excel con todos los programas universitarios de pregrado **activos** en el país.
2. Lo compara contra la descarga de la semana anterior.
3. Detecta tres tipos de novedades: programas **nuevos**, programas que quedaron **inactivos**, y programas que fueron **modificados** (cambiaron de modalidad, créditos, costo de matrícula o de sede).
4. Guarda esos hallazgos acumulados en archivos Excel (para no perder el historial).
5. Envía un correo con el resumen a la lista de personas interesadas.
6. Publica un panel visual (dashboard) en internet con gráficos y tablas filtrables.

Todo esto corre solo, sin que nadie tenga que prender su computador — vive en la nube usando **GitHub Actions** (lo explico en la sección 10).

2. El flujo completo, de principio a fin

Así se ve el proceso completo, en orden, cada vez que se ejecuta:



Cada una de estas cajas se explica a fondo en las secciones siguientes.

3. Mapa de carpetas y archivos

```
snies-monitor/
├── scripts/
│   ├── run_snies.py      → El cerebro: descarga, compara, clasifica, guarda
│   └── send_report.py    → Arma y envía el correo
├── docs/
│   ├── generar_dashboard.py → Todo lo que ve la gente en la página web
│   ├── generar_dashboard.py → Genera todo el HTML del dashboard
│   ├── index.html        → Página principal (SE GENERA SOLA, no editar a mano)
│   ├── nuevos.html       → Detalle de programas nuevos (SE GENERA SOLA)
│   ├── inactivos.html    → Detalle de programas inactivos (SE GENERA SOLA)
│   ├── modificados.html  → Detalle de programas modificados (SE GENERA SOLA)
│   ├── modificados_creditos.html → Análisis a fondo de cambios de créditos
│   ├── modificados_costos.html → Análisis a fondo de cambios de matrícula
│   └── dashboard_data.json → Los datos "crudos" que consume el HTML
├── data/
│   ├── novedades/
│   │   ├── Nuevos_pregrado.xlsx
│   │   ├── Inactivos_pregrado.xlsx
│   │   └── Modificados_pregrado.xlsx
│   └── Categorización divisiones SNIES.xlsx → Tabla que traduce "área de conocimiento SNIES" → "a qué facultad/división de Uninorte compete"
├── Programas/
│   └── Un Excel "foto" (snapshot) por cada corrida, con el nombre "Programas DD-MM-YY.xlsx". Este es el historial crudo, sin procesar.
├── .github/workflows/
│   └── snies_daily.yml → La configuración de "cuándo y cómo correr esto solo"
├── requirements.txt    → Lista de librerías de Python necesarias
└── README.md          → Resumen corto del proyecto
```

Regla de oro: todo lo que está dentro de `docs/*.html` (menos `modificados_creditos.html` y `modificados_costos.html`, que también se generan pero valen la pena mirar aparte) se **reescribe automáticamente** cada vez que corre `generar_dashboard.py`. Si alguien edita `docs/index.html` a mano, ese cambio se pierde en la próxima corrida. Para cambiar cómo se ve la página hay que editar `docs/generar_dashboard.py`, no el HTML final.

4. Paso 1 — Cómo se descarga la información (web scraping)

4.1 ¿Qué es "hacer scraping"?

"Scraping" es la técnica de programar un robot que **usa un navegador de internet como lo usaría una persona**: entra a una página, hace clic en botones, llena filtros, espera a que cargue, y descarga archivos. Se usa cuando la página no ofrece una forma más directa (como una API) de obtener los datos.

Este proyecto usa una librería llamada **Selenium**, que controla una copia real de Google Chrome desde Python. La diferencia con un navegador normal es que corre en modo **"headless"** (sin ventana visible) cuando está en el servidor de GitHub — literalmente no hay pantalla, pero el navegador funciona igual por dentro.

```
# scripts/run_snies.py
opts.add_argument("--headless=new")          # Chrome invisible
opts.add_argument("--window-size=1920,1080") # aunque sea invisible, simula una pantalla grande
```

4.2 La página y los filtros que se aplican

La página que se visita es:

```
SNIES_URL = "https://hecaa.mineducacion.gov.co/consultaspublicas/programas"
```

Esa página del Ministerio permite filtrar los programas por varios criterios usando **botones de radio** (los circulitos donde solo puedes elegir una opción). El robot selecciona, en este orden, estos cuatro filtros:

FILTRO QUE HACE CLIC	QUÉ SIGNIFICA	POR QUÉ SE USA
"Activo" (institución)	La institución educativa está activa (no cerrada/liquidada)	No tiene sentido vigilar universidades que ya no existen
"Activo (" (programa)	El programa académico específico está activo (no cancelado)	Solo interesan programas que se pueden ofertar hoy
"Pregrado ("	Nivel de formación = pregrado (no posgrado)	Este pipeline solo vigila pregrado por ahora
"Universitario ("	Tipo de programa universitario (excluye técnico/tecnológico)	Uninorte compite en el nivel universitario

En código, esto se ve así (simplificado):

```
_pf_select_radio(driver, "Activo", exact=True)    # institución activa
_wait_ajax(driver)
_pf_select_radio(driver, "Activo ("               # programa activo
_wait_ajax(driver)
_pf_select_radio(driver, "Pregrado ("             # nivel pregrado
_wait_ajax(driver)
_pf_select_radio(driver, "Universitario ("        # tipo universitario
_wait_ajax(driver)
```

Cada vez que se hace clic en un filtro, la página del SNIES recalcula la lista de programas por detrás usando una tecnología llamada **AJAX** (le pide datos nuevos al servidor sin recargar toda la página). Por eso después de cada clic el robot llama a `_wait_ajax(driver)`, que **espera pacientemente a que la página termine de "pensar"** antes de seguir con el siguiente filtro. Si no se esperara, el robot podría hacer clic en el filtro siguiente antes de que el anterior haya surtido efecto, y la descarga saldría mal.

¿Por qué se buscan los botones por su texto ("Activo", "Pregrado") y no por un identificador técnico?

Las páginas de sistemas del gobierno colombiano suelen estar hechas con una tecnología llamada JSF/PrimeFaces, que le pone a cada botón un identificador (`id`) que cambia cada vez que el Ministerio actualiza el portal. Si el robot buscara por ese `id`, dejaría de funcionar con cualquier actualización menor de la página. En cambio, el texto visible ("Activo", "Pregrado") es mucho menos probable que cambie, así que el robot es más resistente a rediseños del portal.

Una vez aplicados los 4 filtros, el robot hace clic en el botón de descarga:

```
XPATHS = {
    "descarga": '//*[@button[.//span[normalize-space()="Descargar programas"]]]',
}
```

Esto es un **XPath**: una forma de decirle al navegador "búscame el botón que contiene un texto que diga exactamente 'Descargar programas'". Igual que con los filtros, se usa el texto visible en vez de un `id` interno, por la misma razón de robustez.

4.3 Esperar la descarga

Después de pedir la descarga, el robot no sabe de antemano cuánto se va a demorar el servidor del gobierno en generar el archivo. Por eso hace lo siguiente:

```
DOWNLOAD_TIMEOUT = 120 # segundos máximos esperando la descarga

elapsed = 0
while elapsed < DOWNLOAD_TIMEOUT:
    time.sleep(5)
    elapsed += 5
    if expected_file.exists() and not partial_file.exists():
        break # ¡ya llegó completo!
else:
    raise TimeoutError("Archivo no apareció tras 120s...")
```

Revisa cada 5 segundos si ya apareció el archivo `Programas.xlsx` completo (mientras se está descargando, Chrome lo llama temporalmente `Programas.crdownload`). Si pasan 2 minutos sin que aparezca, el robot se rinde y reporta el error — normalmente esto pasa si el portal del gobierno está caído o cambió de diseño.

4.4 Capturas de pantalla para depurar

Como el navegador corre invisible en el servidor de GitHub, es imposible "verlo" en vivo si algo sale mal. Por eso el robot se toma dos fotos (capturas de pantalla) durante el proceso y las guarda en `tmp/`:

```
driver.save_screenshot(str(TMP_DIR / "debug_snies.png")) # justo al abrir la página
driver.save_screenshot(str(TMP_DIR / "debug_post_filtros.png")) # después de aplicar los filtros
```

Estas capturas no se guardan en el repositorio (la carpeta `tmp/` está en `.gitignore`), pero si el workflow falla puedes descargarlas desde la pestaña **Actions** de GitHub como "artifacts" del run que falló, para ver exactamente qué estaba mostrando la página en ese momento.

4.5 El "candado de seguridad" contra descargas corruptas

Antes de aceptar el Excel descargado como válido, el pipeline hace una validación de sentido común: un archivo de **pregrado universitario activo** en Colombia normalmente tiene unos 5.000 programas. Si el archivo descargado tiene más de 10.000 filas, es señal de que **algo salió mal con los filtros** (por ejemplo, se descargó sin filtrar, con posgrados incluidos, etc.):

```
UMBRAL = 10_000
if len(df_hoy) > UMBRAL:
    log.error(f"Snapshot HOY tiene {len(df_hoy)} programas — demasiados. "
            "Probable descarga sin filtros. Abortando comparación.")
    return vacio # no compara nada, no daña el historial acumulado
```

Si esto pasa, el pipeline **no** compara ni acumula nada ese día — mejor no reportar novedades que reportar cientos de falsos positivos por una descarga corrupta.

5. Paso 2 — Qué información se guarda de cada programa

Del Excel descargado, el sistema no guarda todas las columnas — solo estas, que son las relevantes para el monitoreo:

COLUMNA	QUÉ ES, EN PALABRAS SIMPLES
CÓDIGO_INSTITUCIÓN	Número único que identifica a la universidad
NOMBRE_INSTITUCIÓN	Nombre de la universidad
SECTOR	Oficial (pública) o Privado
DEPARTAMENTO_OFERTA_PROGRAMA	En qué departamento de Colombia se dicta el programa
MUNICIPIO_OFERTA_PROGRAMA	En qué ciudad/municipio se dicta el programa
CÓDIGO_SNIES_DEL_PROGRAMA	El identificador único del programa. Es como la "cédula" de cada programa — nunca cambia mientras el programa exista. Todo el sistema de comparación se basa en este número.
NOMBRE_DEL_PROGRAMA	Ej: "CONTADURÍA PÚBLICA"
MODALIDAD	Presencial, Virtual, A distancia, Dual, o combinaciones ("Híbrida...")
NÚMERO_CRÉDITOS	Cuántos créditos académicos tiene el programa
NÚMERO_PERIODOS_DE_DURACIÓN	Cuántos semestres/periodos dura la carrera
PERIODICIDAD	Semestral, anual, etc.
COSTO_MATRÍCULA_ESTUD_NUEVOS	Valor de la matrícula para estudiantes que ingresan por primera vez
PERIODICIDAD_ADMISIONES	Cada cuánto abre admisiones el programa
FECHA_DE_REGISTRO_EN_SNIES	Cuándo se registró el programa en el sistema del Ministerio
CINE_F_2013_AC_CAMPO_AMPLIO / ..._ESPECÍFIC / ..._DETALLADO	Clasificación internacional de la UNESCO por área del conocimiento (Educación, Salud, Ingeniería, etc.), en 3 niveles de detalle
NÚCLEO_BÁSICO_DEL_CONOCIMIENTO	Otra clasificación de áreas de conocimiento, propia de Colombia

Con la columna CINE_F_2013_AC_CAMPO_DETALLADO se hace un cruce contra el archivo data/Categorización divisiones SNIES.xlsx (hoja Hoja3), que traduce cada área de conocimiento a "**¿qué facultad/división de Uninorte compete con este programa?**" (columna DIVISIÓN UNINORTE). Así, cuando aparece un programa nuevo de Ingeniería en otra universidad, el reporte ya te dice directamente si eso le compete a la División de Ingenierías de Uninorte, sin que tengas que buscarlo tú.

Además, al final se le limpian los datos: se quitan las dos últimas filas del Excel (que son un aviso legal del SNIES, no un programa real), y se convierten los tipos de dato (el código SNIES a número entero, la fecha de registro a fecha real, los créditos a número).

6. Paso 3 — Cómo se decide si un programa es Nuevo, Inactivo o Modificado

6.1 La idea de "snapshot" (foto) y el código SNIES como huella digital

Cada vez que el robot descarga el Excel, esa descarga es una **"foto" (snapshot)** del estado del SNIES en ese momento exacto. El sistema guarda esa foto permanentemente en `Programas/Programas DD-MM-YY.xlsx` y luego la compara contra **la foto anterior más reciente** que exista en esa carpeta.

Como cada programa tiene un `CÓDIGO_SNIES_DEL_PROGRAMA` único que nunca cambia, comparar dos fotos es tan simple como comparar dos listas de códigos:

```
snies_hoy = set(df_hoy["CÓDIGO_SNIES_DEL_PROGRAMA"]) # códigos que aparecen HOY
snies_ant = set(df_ant["CÓDIGO_SNIES_DEL_PROGRAMA"]) # códigos que aparecían ANTES
```

6.2 Programa Nuevo

Un programa es "Nuevo" si su código aparece en la foto de hoy, pero **no** aparecía en la foto anterior:

```
nuevosDF = df_hoy[df_hoy["CÓDIGO_SNIES_DEL_PROGRAMA"].isin(snies_hoy - snies_ant)]
```

En español: "todos los programas de hoy cuyo código no estaba en la lista de antes".

Ejemplo real que capturó el sistema:

Apareció el 11/05/2026 la **Licenciatura en Matemáticas** de la Corporación Universitaria Antonio José de Sucre (CORPOSUCRE), modalidad Presencial, en Sucre. El sistema la clasificó como del área "Educación" (CINE) y la mapeó a la división "Instituto de Estudios en Educación" de Uninorte.

6.3 Programa Inactivo

Es el caso contrario: el código estaba en la foto anterior, pero **ya no aparece** en la de hoy.

```
inactivosDF = df_ant[df_ant["CÓDIGO_SNIES_DEL_PROGRAMA"].isin(snies_ant - snies_hoy)]
```

Ojo: como el filtro de descarga solo trae programas **activos**, un programa puede "desaparecer" de la lista tanto porque lo cerraron de verdad, como porque el Ministerio lo pasó a estado inactivo/suspendido en el SNIES. Para el reporte, ambos casos se ven igual: dejó de estar activo.

6.4 Programa Modificado — las 4 variables que se vigilan

Este es el caso más interesante. Un programa se marca como "Modificado" cuando su código existe **en ambas fotos** (hoy y antes — o sea, sigue activo) **pero al menos uno** de estos 4 campos cambió de valor:

```
COLS_VIGILAR = [
    "MODALIDAD",
    "NÚMERO_CRÉDITOS",
    "COSTO_MATRÍCULA_ESTUD_NUEVOS",
    "MUNICIPIO_OFERTA_PROGRAMA",
]
```

CAMPO VIGILADO	QUÉ SIGNIFICA QUE CAMBIE
MODALIDAD	El programa pasó de Presencial a Virtual, o agregó una modalidad híbrida, etc.
NÚMERO_CRÉDITOS	Le subieron o bajaron los créditos académicos (ej: de 150 a 144)
COSTO_MATRÍCULA_ESTUD_NUEVOS	Cambió el valor de la matrícula para estudiantes nuevos
MUNICIPIO_OFERTA_PROGRAMA	El programa se empezó a ofrecer en otra ciudad, o cambió de sede

Para cada programa común a ambas fotos, el sistema arma una tabla donde pone lado a lado el valor de hoy y el valor de antes (usando sufijos `_NUEVO` y `_ANTIGUO`), y marca como "modificado" cualquier fila donde alguno de esos 4 pares sea diferente:

```
mascara = pd.Series(False, index=comparativa.index)
for col in COLS_VIGILAR:
    col_n, col_a = f"{col}_NUEVO", f"{col}_ANTIGUO"
    mascara |= (comparativa[col_n].astype(str) != comparativa[col_a].astype(str))

modificadosDF = comparativa[mascara]
```

Además, el sistema genera automáticamente una columna `QUE_CAMBIO` que explica en texto legible **exactamente qué cambió y de qué valor a qué valor**:

```
def _que_cambio(row):
    partes = []
    for col in COLS_VIGILAR:
        val_n, val_a = str(row[f"{col}_NUEVO"]), str(row[f"{col}_ANTIGUO"])
        if val_n != val_a:
            partes.append(f"{col}: {val_a} → {val_n}")
    return " | ".join(partes)
```

Ejemplo real capturado por el sistema:

Contaduría Pública de UNIMINUTO (Casanare) apareció como modificada, con:

`QUE_CAMBIO = "NÚMERO_CRÉDITOS: 150 → 144"`

Es decir: le bajaron 6 créditos al programa. Si además hubieran cambiado la modalidad ese mismo periodo, el texto se vería así:

`"NÚMERO_CRÉDITOS: 150 → 144 | MODALIDAD: Presencial → Presencial-Virtual"`

6.5 ⚠️ Importante: qué cambios el sistema NO detecta

Esto es clave para entender los límites del monitor. **Solo se vigilan esos 4 campos**. Si un programa cambia de **nombre**, de **institución dueña**, de **duración en periodos**, de **núcleo básico del conocimiento**, o de cualquier otra columna que no esté en `COLS_VIGILAR`, el sistema **no lo va a reportar como "Modificado"** — para el comparador, ese programa sigue "igual", aunque en la realidad algo haya cambiado.

Esto es una decisión de diseño (probablemente para no generar demasiado ruido con cambios menores de redacción), pero es importante que quien lea los reportes sepa que existe este punto ciego. Si en algún momento se quiere vigilar un campo adicional (por ejemplo, la duración en periodos), basta con agregarlo a la lista `COLS_VIGILAR` en `scripts/run_snies.py` (línea 89) — no hay que tocar nada más, el resto del sistema (correo, dashboard) ya sabe leer cualquier campo que aparezca en `QUE_CAMBIO`.

6.6 Una protección extra: evitar comparaciones "fantasma"

A veces el portal del SNIES devuelve accidentalmente el mismo código de programa dos veces en un mismo archivo (por errores del portal, o por variantes de modalidad sin código distinto). Si eso pasara y no se controlara, al cruzar la foto de hoy con la de antes se generarían combinaciones falsas (por ejemplo, 2 filas de hoy $\times$ 3 filas de antes = 6 "modificaciones" en vez de 1). El código detecta esos duplicados, avisa por log, y se queda solo con la primera aparición de cada código antes de comparar — así nunca se inventan modificaciones que no existen.

7. Paso 4 — Cómo se guarda el historial (acumulación en Excel)

Cada corrida **no reemplaza** los resultados anteriores — los **acumula**. Los tres archivos en `data/novedades/` (`Nuevos_pregrado.xlsx`, `Inactivos_pregrado.xlsx`, `Modificados_pregrado.xlsx`) van creciendo semana tras semana: hoy tienen, por ejemplo, 686 programas nuevos y 1.253 modificaciones acumuladas desde que arrancó el monitoreo.

```
def acumular(existing_path, nuevo_df):
    dedup_cols = ["CÓDIGO_SNIES_DEL_PROGRAMA", "FECHA_OBTENCION"]
    existing = pd.read_excel(existing_path)
    combined = pd.concat([existing, nuevo_df], ignore_index=True)
    return combined.drop_duplicates(subset=dedup_cols, keep="last")
```

La deduplicación es por `CÓDIGO_SNIES_DEL_PROGRAMA` + `FECHA_OBTENCION` (la fecha en que se detectó la novedad): esto evita que si el robot corre dos veces el mismo día, se dupliquen las filas — pero si el mismo programa vuelve a modificarse en una semana distinta, sí queda registrado de nuevo (con otra fecha), porque es información nueva.

Aparte de esos 3 archivos "resumen de novedades", **cada corrida también guarda una copia completa y sin procesar** del Excel descargado ese día en `Programas/Programas DD-MM-YY.xlsx`. Esa carpeta es el verdadero historial crudo — de ahí sale el gráfico de "evolución del total de programas activos" del dashboard, y es contra ese historial que se calcula, cada semana, cuál es "la foto anterior" para comparar.

Nota práctica: si por cualquier motivo hay que reconstruir el historial de "novedades" desde cero, basta con borrar los archivos de `data/novedades/` (o dejar solo los que se quieran conservar) — el pipeline los vuelve a crear automáticamente la próxima vez que corra, usando lo que compare desde `Programas/` en adelante. **No se debe borrar la carpeta `Programas/`** salvo que se quiera perder el historial completo, porque de ahí es de donde sale absolutamente toda la comparación y todos los gráficos históricos del dashboard.

8. Paso 5 — El correo automático

Al final de cada corrida, `scripts/run_snies.py` llama a `send_report.py`, que construye un correo en formato HTML (con colores, tablas, y un botón) y lo envía usando el protocolo **SMTP** (el protocolo estándar para enviar correos por programación).

8.1 Qué contiene el correo

- Un título con la fecha del reporte.
- Un botón grande que enlaza directo al dashboard.
- Una sección  **Programas Nuevos**, con una tabla de **todos** los nuevos de esa corrida (nombre, institución, ciudad, división Uninorte).
- Una sección  **Programas Inactivos**, con los primeros 10 (si hay más, dice "... y N registro(s) más en el adjunto Excel").
- Una sección  **Programas Modificados**, también con los primeros 10 y el detalle de `QUE_CAMBIO`.
- Los tres archivos Excel de `data/novedades/` **adjuntos completos** (para quien quiera ver todo el detalle, no solo la vista previa).

```
# scripts/send_report.py
adjuntos = sorted(NOVEDADES_DIR.glob("*_pregrado.xlsx"))
for path in adjuntos:
    # ... adjunta cada Excel al correo
```

8.2 De dónde saca las credenciales

El correo **nunca tiene contraseñas escritas en el código**. Las lee de variables de entorno (información que se le pasa al programa desde afuera, no desde el archivo mismo) que en producción vienen de los *Secrets* de GitHub (sección 11):

```
smtp_user    = os.environ["SMTP_USER"]      # ej: monitor.snies@gmail.com
smtp_pass    = os.environ["SMTP_PASS"]      # contraseña de aplicación
destinatarios = os.environ["DESTINARIOS"].split(",") # lista separada por comas
```

8.3 Por dónde sale el correo

```
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587
```

Actualmente el correo sale por **Gmail** (aunque hay un comentario en el código que dice "Office 365" — es un comentario desactualizado de una configuración anterior, ver sección 17). Se conecta, se autentica con usuario/contraseña, y envía.

8.4 Si algo falla, no se cae todo el proceso

El envío de correo está protegido con un manejo de errores: si por lo que sea el correo no se pudo enviar (contraseña vencida, Gmail bloqueó el acceso, etc.), el error se registra en el log del workflow **pero el resto del pipeline sigue** — los datos ya se guardaron en `data/novedades/` y `Programas/` de todas formas, y el dashboard se genera igual. Es decir: nunca se pierde información por un fallo de correo, en el peor caso solo no llega el aviso por email esa semana (pero el dashboard sí se actualiza).

9. Paso 6 — El Dashboard (la página web)

Todo lo que ves en https://dirplaneacionun.github.io/snies_monitor/ se genera automáticamente con `docs/generar_dashboard.py`, que lee **todo** el historial (`Programas/` + `data/novedades/`) y escribe archivos HTML nuevos cada vez. No hay una base de datos ni un servidor corriendo detrás — son páginas HTML "estáticas" con los datos incrustados directamente adentro (en formato JSON), y todo el filtrado/gráficas pasan en el navegador de quien lo visita, con JavaScript. Por eso la página carga rápido y no necesita "backend".

9.1 Página principal (`index.html`)

Lo primero que se ve son 4 tarjetas (KPIs = indicadores clave):

- **Programas activos:** el total de programas universitarios de pregrado activos hoy en Colombia (según la última foto descargada).
- **Nuevos (último run) / Inactivos (último run) / Modificados (último run):** cuántos se detectaron en la corrida más reciente, con el acumulado histórico debajo. Cada tarjeta es un botón que te lleva a su página de detalle.

Debajo hay una **barra de filtros globales**, fija en la parte de arriba de la pantalla, que afecta *todos* los gráficos y tablas de la página a la vez:

```
<input id="gf-q" placeholder="Buscar por nombre, institución, código SNIES...">
<select id="gf-sector">Todos los sectores</select>
<select id="gf-depto">Todos los departamentos</select>
<select id="gf-modalidad">Todas las modalidades</select>
```

- El cuadro de texto busca por coincidencia parcial en **cualquier columna** de cada fila (nombre del programa, institución, código, etc.), ignorando tildes y mayúsculas/minúsculas, y soporta varias palabras a la vez (todas deben aparecer, en cualquier orden).
- Los combos de Sector / Departamento / Modalidad filtran por coincidencia exacta.
- La "Modalidad" en este filtro global está **agrupada**: solo se listan las 6 modalidades más comunes por nombre propio, y todo lo demás se agrupa bajo "Otras" — esto es para que el mismo filtro tenga sentido tanto en la tabla de novedades como en el gráfico de evolución histórica (que por espacio no puede mostrar 11 modalidades distintas).

Después de los KPIs vienen los gráficos (hechos con una librería llamada **Plotly**, que dibuja gráficos interactivos: puedes pasar el mouse para ver el valor exacto, hacer zoom, etc.):

GRÁFICO	QUÉ MUESTRA
Total acumulado de programas activos	Línea con la evolución del número total de programas activos en el país, corrida tras corrida
Aperturas vs. cierres netos	Barras verdes (nuevos) contra barras rojas (inactivos) por periodo, con una línea del neto. Si un periodo tuvo muy pocas corridas del monitor (menos de 5), se marca con una advertencia porque esos números no son comparables 1:1
Evolución de la modalidad	Una tarjeta con mini-gráfico por cada modalidad (Presencial, Virtual, etc.), ordenadas de mayor a menor crecimiento porcentual
Por sector	Torta Oficial vs. Privado
Top 15 departamentos de oferta	Barras horizontales
Distribución por duración (periodos)	Barras apiladas por periodicidad — si haces clic en una barra , se abre abajo una tabla con el listado exacto de programas activos de esa duración

Más abajo están las **pestañas** (tabs) de Nuevos / Inactivos / Modificados, cada una con su tabla — se puede ordenar cualquier columna haciendo clic en su encabezado (una flecha ↕ indica que es ordenable).

9.2 Páginas de detalle (`nuevos.html` , `inactivos.html` , `modificados.html`)

Cuando haces clic en una de las tarjetas KPI o en una pestaña, llegas a una página dedicada con **todo** el historial acumulado de ese tipo de novedad (no solo la última corrida), con su propia barra de filtros (texto, sector, departamento, institución con autocompletar, división Uninorte o campo CINE, y fecha), más gráficos propios de ese tipo:

- Distribución por sector, top 10 instituciones, por división Uninorte, por modalidad.
- Top 15 departamentos afectados.
- Un gráfico acumulado por campo CINE a través del tiempo, donde puedes **buscar y agregar** campos CINE específicos para comparar su evolución lado a lado (útil para preguntas del tipo "¿cómo ha crecido la oferta de Ingeniería de Sistemas en el país?").
- En `modificados.html` además hay un gráfico de "tipo de cambio detectado" (cuántas modificaciones fueron de modalidad vs. créditos vs. costo vs. municipio) y un dispersograma (scatter) de créditos antes vs. después.

9.3 Páginas de análisis profundo: créditos y costos

`modificados_creditos.html` y `modificados_costos.html` son páginas dedicadas exclusivamente a analizar, respectivamente, los cambios de número de créditos y los cambios de costo de matrícula. Incluyen:

- Rankings de instituciones/áreas con más cambios.
- Un histograma de cuánto suben o bajan (para costos se usa el **cambio porcentual**, no el valor en pesos — esto es a propósito: no es lo mismo que una matrícula de 300 mil suba 50 mil, a que una de 20 millones suba 50 mil; el porcentaje sí es comparable entre programas de valores muy distintos).
- Análisis de "co-cambios": si un programa cambió de créditos, ¿también cambió de costo o de modalidad al mismo tiempo? Esto ayuda a distinguir un simple ajuste curricular de una reestructuración completa del programa.
- Un dispersograma de "antes" vs. "después".

9.4 Cómo y cuándo se regenera

El dashboard se regenera **automáticamente** en cada corrida del workflow, justo después de que se guardan los datos nuevos (ver el paso "Generar dashboard" en la sección 10.4). Nunca hay que generarlo a mano en producción — solo tendrías que correrlo tú mismo si quieres **probar cambios en el diseño** antes de subirlos (ver sección 15).

10. GitHub Actions: el robot que hace todo esto solo

10.1 ¿Qué es un "workflow", un "cron" y un "secret"?

- **GitHub Actions** es un servicio gratuito (dentro de ciertos límites) de GitHub que te deja programar tareas automáticas que corren en un computador temporal en la nube ("el runner"), cada vez que pasa algo (alguien hace push, se cumple un horario, etc.).
- Un **workflow** es un archivo `.yaml` que describe, paso a paso, qué se debe hacer. En este proyecto es `.github/workflows/snies_daily.yaml`.
- Un **cron** es la forma estándar (viene de Unix, de hace más de 40 años) de escribir "a qué hora y qué días correr algo", con 5 números/símbolos separados por espacio: `minuto hora día-del-mes mes día-de-la-semana`.
- Un **secret** es un valor sensible (contraseña, token) que GitHub guarda cifrado y que el workflow puede usar sin que quede visible en el código ni en los logs (ver sección 11).

10.2 El cronograma actual y cómo cambiarlo

```
on:
  schedule:
    - cron: '0 13 * * 2' # 08:00 hora Colombia (UTC-5), martes
  workflow_dispatch:
```

Esto se lee así: minuto `0`, hora `13` (en UTC — hora universal), cualquier día del mes (`*`), cualquier mes (`*`), día de la semana `2` (martes, donde domingo=0, lunes=1, martes=2...). Colombia está en UTC-5 todo el año (no cambia de horario), así que 13:00 UTC = **8:00 a.m. hora Colombia**, todos los **martes**.

`workflow_dispatch` es lo que permite correrlo **manualmente** desde la pestaña Actions de GitHub, en cualquier momento, sin esperar al martes (ver 10.3).

Si en algún momento se necesita cambiar la frecuencia (por ejemplo, volver a diario, o cambiar de día), solo hay que editar esa línea de cron. Algunos ejemplos:

```
- cron: '0 13 * * *' # todos los días, 8am Colombia
- cron: '0 13 * * 1-5' # lunes a viernes, 8am Colombia
- cron: '0 13 * * 1' # todos los lunes, 8am Colombia
- cron: '0 13 1 * *' # el día 1 de cada mes, 8am Colombia
```

Después de cambiar el cron, hay que hacer commit y push del archivo — GitHub vuelve a leer el horario automáticamente, no hay que "reactivar" nada aparte.

Ojo: GitHub Actions no garantiza que el cron dispare exactamente al segundo — en momentos de mucha carga en los servidores de GitHub puede haber un retraso de varios minutos. Es normal.

10.3 Cómo correrlo manualmente

Si no quieres esperar al próximo martes para probar un cambio:

1. Entra al repositorio en GitHub: `https://github.com/dirplaneacionun/snies_monitor`
2. Pestaña **Actions** (arriba).
3. En la lista de la izquierda, clic en **SNIES Monitor**.
4. Botón **Run workflow** (a la derecha) → **Run workflow** de nuevo para confirmar.
5. Se abre una corrida nueva; puedes hacer clic en ella para ver los logs en vivo, paso a paso.

Esto es exactamente lo mismo que pasaría un martes a las 8am — corre el pipeline completo, envía el correo real, y publica el dashboard real. **No es un modo de "prueba" que no tenga efecto** — si corres esto manualmente, sí le llega correo a los destinatarios de verdad y sí se actualiza la página pública.

10.4 Qué hace cada paso del workflow

```
jobs:
  run-snies:
    runs-on: ubuntu-latest      # el "computador temporal" es Linux
    timeout-minutes: 60        # si se demora más de 1 hora, se cancela solo

    steps:
      - name: Checkout          # 1. Descarga el código del repositorio
      - name: Instalar Chrome    # 2. Instala Google Chrome (necesario para Selenium)
      - name: Setup Python       # 3. Instala Python 3.11
      - name: Instalar dependencias # 4. pip install -r requirements.txt
      - name: Ejecutar pipeline   # 5. python scripts/run_snies.py (descarga+compara+correo)
      - name: Committear cambios  # 6. Guarda data/ y Programas/ en el repositorio
      - name: Generar dashboard   # 7. python docs/generar_dashboard.py
      - name: Publicar dashboard  # 8. Guarda docs/ en el repositorio
```

Los pasos 6 y 8 son commits **separados** — por eso en el historial de git ves dos mensajes distintos el mismo día, por ejemplo:

```
dashboard: 2026-07-02
snies: 2026-07-02
```

El primer commit (`snies: FECHA`) guarda los datos crudos y las novedades; el segundo (`dashboard: FECHA`) guarda el HTML ya regenerado del dashboard. Si algún día no hay cambios reales que guardar (por ejemplo, si el pipeline falló antes de llegar ahí), el paso simplemente no crea un commit vacío — usa `git diff --staged --quiet || git commit ...`, que en español dice "si no hay nada nuevo, no hagas commit".

Requisito importante para que el `git push` funcione: el repositorio debe tener activado, en `Settings → Actions → General → Workflow permissions`, la opción **"Read and write permissions"**. Si esa opción está en modo solo lectura, todo el pipeline corre bien (descarga, compara, envía el correo) pero el paso de guardar los cambios fallará con un error de permisos al final. Si algún día ves que el correo llega pero el dashboard no se actualiza, este es el primer lugar para revisar.

11. Los "Secrets" de GitHub

Los *Secrets* son la forma en que GitHub Actions maneja información sensible (contraseñas, tokens) sin que quede expuesta en el código fuente, que es público. Un secret se define una sola vez desde la configuración del repositorio, y el workflow lo referencia con `${{ secrets.NOMBRE }}` sin que su valor real aparezca nunca en los logs (GitHub lo enmascara automáticamente si por error se intentara imprimir).

Este proyecto usa 3 secrets propios, más uno automático que provee GitHub:

SECRET	PARA QUÉ SE USA	¿AUTOMÁTICO?
SMTP_USER	El correo que envía los reportes (ej. <code>monitor.snies@gmail.com</code>)	No, hay que crearlo
SMTP_PASS	La contraseña (de aplicación) de esa cuenta de correo	No, hay que crearlo
DESTINATARIOS	Lista de correos que reciben el reporte, separados por coma	No, hay que crearlo
GITHUB_TOKEN	Un token temporal que usa el workflow para poder hacer <code>git push</code> de vuelta al repositorio	Sí , GitHub lo genera solo en cada corrida — no hay que crearlo ni renovarlo nunca

Cómo ver o cambiar un secret

1. Entra a `https://github.com/dirplaneacionun/snies_monitor` (necesitas permisos de administrador o de escritura en el repositorio — si no te deja entrar a Settings, pide que alguien con acceso lo haga o te dé permisos).
2. Pestaña **Settings**.
3. En el menú de la izquierda: **Secrets and variables** → **Actions**.
4. Ahí verás la lista: `SMTP_USER` , `SMTP_PASS` , `DESTINATARIOS` .

Importante: por seguridad, GitHub nunca te deja ver el valor actual de un secret una vez guardado — solo puedes **sobrescribirlo** con un valor nuevo (botón "Update"). Si no sabes qué correo o contraseña hay configurados actualmente, no hay forma de consultarlo desde la interfaz; hay que preguntarle a quien lo haya configurado, o simplemente definir uno nuevo.

No hace falta tocar ni un archivo de código ni redesplegar nada al cambiar un secret — la próxima vez que el workflow corra (martes siguiente, o manualmente), ya usará el valor nuevo automáticamente.

12. Cómo cambiar los destinatarios del correo

1. Ve a `Settings` → `Secrets and variables` → `Actions` en el repositorio (ver sección 11).
2. Busca el secret `DESTINATARIOS` y haz clic en el lápiz/**Update**.
3. Escribe la lista completa de correos separados por comas. No importa si dejas espacios después de la coma — el código les hace `strip()` (les quita los espacios sobrantes) automáticamente:

```
persona1@uninorte.edu.co, persona2@uninorte.edu.co, decano.negocios@uninorte.edu.co
```

4. Guarda ("Update secret").

Eso es todo — **la lista se reemplaza completa**, no se "agrega" un correo a lo que ya había. Si quieres agregar una persona sin quitar a nadie, tienes que escribir la lista completa de nuevo (las que ya estaban + la nueva).

Recuerda: este cambio no se ve reflejado hasta la próxima corrida del workflow (el próximo martes 8am, o antes si alguien lo corre manualmente).

13. Cómo cambiar la cuenta de correo que envía los reportes

Este es el procedimiento completo para el día en que haya que migrar a otra cuenta de Gmail (por ejemplo, porque la cuenta actual se va a dar de baja, o porque cambia quién administra el monitor).

13.1 Si sigues usando Gmail (caso más común)

Gmail **no permite** que un programa inicie sesión con la contraseña normal de la cuenta por seguridad — hay que generar una "**contraseña de aplicación**" (App Password), que es una clave especial de 16 caracteres solo para este uso.

1. Entra a la cuenta de Gmail que vas a usar para enviar los reportes.
2. Activa la **verificación en dos pasos** si no la tiene activada (obligatorio: sin esto, Google ni siquiera muestra la opción de contraseñas de aplicación). Se hace desde `myaccount.google.com` → Seguridad .
3. Ve a `myaccount.google.com/apppasswords` (o busca "Contraseñas de aplicaciones" dentro de Seguridad).
4. Crea una nueva, ponle un nombre que la identifique (ej: "SNIES Monitor GitHub Actions").
5. Google te muestra una clave de 16 caracteres (sin espacios) — **cópiala en ese momento**, porque después no se puede volver a ver.
6. En GitHub, ve a `Settings` → `Secrets and variables` → `Actions` (sección 11) y actualiza:
 - `SMTP_USER` → el nuevo correo completo (ej. `nuevo.monitor@gmail.com`)
 - `SMTP_PASS` → la contraseña de aplicación de 16 caracteres que acabas de generar (NO la contraseña normal de la cuenta)
7. Listo — no hay que tocar código, porque el host (`smtp.gmail.com`) y el puerto (`587`) siguen siendo los mismos.

13.2 Si te cambias a un proveedor distinto de Gmail (Outlook/Office 365, correo institucional propio, etc.)

Aquí sí hay que editar código, porque el servidor SMTP está escrito directamente en `scripts/send_report.py` :

```
# scripts/send_report.py, líneas 30-31
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587
```

Pasos:

1. Averigua el host y puerto SMTP del nuevo proveedor. Algunos ejemplos comunes:

PROVEEDOR	HOST	PUERTO
Gmail	<code>smtp.gmail.com</code>	<code>587</code>
Outlook / Office 365	<code>smtp.office365.com</code>	<code>587</code>
Zoho Mail	<code>smtp.zoho.com</code>	<code>587</code>

2. Edita esas dos líneas en `scripts/send_report.py` con el host/puerto correcto.
3. Haz commit y push del cambio (esto sí requiere subir código, a diferencia de un cambio de secret).

4. Actualiza los secrets `SMTP_USER` y `SMTP_PASS` en GitHub con las credenciales de la cuenta nueva (sección 11). Ojo: algunos proveedores (como Office 365) también pueden requerir activar "SMTP AUTH" en la configuración de la cuenta antes de que funcione — revisa la documentación de tu proveedor si el envío falla con un error de autenticación.

13.3 Cómo probar que quedó bien configurado

La forma más simple es correr el workflow manualmente (sección 10.3) y revisar el log del paso "Ejecutar pipeline" — si algo falla en el envío, vas a ver algo como:

```
ERROR ... Error enviando el correo.  
smtpplib.SMTPAuthenticationError: (535, b'5.7.8 Username and Password not accepted...')
```

Ese error específico casi siempre significa: usaste la contraseña normal de Gmail en vez de una contraseña de aplicación, o el usuario/contraseña están mal escritos en los secrets.

Si prefieres probar sin esperar a que corra el scraping completo (que se demora varios minutos), puedes correrlo localmente con datos falsos — ver sección 15.

14. GitHub Pages: cómo se publica la página web

GitHub Pages es un servicio gratuito de GitHub que convierte archivos HTML de tu repositorio en un sitio web público, sin necesidad de contratar hosting. Este proyecto lo usa para publicar el dashboard.

Cómo verificar/activar la configuración

1. `Settings` → `Pages` en el repositorio.
2. En **"Source"** debe estar seleccionado: **Deploy from a branch**.
3. En **"Branch"**: `main` y la carpeta `/docs`.
4. Guarda si hiciste algún cambio.

Con eso, **cualquier archivo dentro de `docs/`** que se suba a la rama `main` queda publicado automáticamente en `https://dirplaneacionun.github.io/snies_monitor/` en cuestión de 1-2 minutos. No hay que hacer nada más — el paso "Publicar dashboard" del workflow (sección 10.4) ya se encarga de subir la carpeta `docs/` actualizada cada semana.

Si algún día se quiere mover el proyecto a otra organización/cuenta de GitHub, o renombrar el repositorio, la URL del dashboard cambiaría — y hay que actualizarla en **dos lugares del código** además del propio README:

```
# scripts/send_report.py
DASHBOARD_URL = "https://dirplaneacionun.github.io/snies_monitor/"
```

y en el badge del `README.md`.

15. Cómo correr todo esto en tu propio computador

Sirve para probar cambios antes de subirlos, o para depurar un problema sin tener que esperar al workflow de GitHub.

15.1 Preparar el entorno

```
# Clona el repositorio si aún no lo tienes
git clone https://github.com/dirplaneacionun/snies_monitor.git
cd snies_monitor

# Instala las librerías de Python necesarias
pip install -r requirements.txt
```

Las librerías usadas (`requirements.txt`) son:

LIBRERÍA	PARA QUÉ
pandas	Leer/escribir/comparar los Excel (es la herramienta estándar de Python para manejar tablas de datos)
openpyxl	El "motor" que usa pandas por detrás para leer/escribir archivos <code>.xlsx</code>
selenium	Controlar el navegador Chrome automáticamente
webdriver-manager	Descarga automáticamente la versión correcta de ChromeDriver (el "conector" entre Selenium y Chrome) — no hay que instalarlo a mano

15.2 Correr solo la descarga + comparación (sin correo)

Si no configuras las variables de entorno del correo, el script sigue funcionando — solo falla (sin romper nada) al intentar enviar el correo al final, y lo indica en el log:

```
python scripts/run_snies.py
```

15.3 Correr todo, incluyendo el correo real

```
# En Windows (PowerShell):
$env:SMTP_USER = "tu_correo@gmail.com"
$env:SMTP_PASS = "tu_contraseña_de_aplicacion"
$env:DESTINARIOS = "tu_correo_de_prueba@gmail.com"
python scripts/run_snies.py
```

⚠ **Ten cuidado:** esto envía un correo real a los destinatarios reales que hayas puesto. Si solo quieres probar el envío, pon tu propio correo en `DESTINARIOS` en vez de la lista real de la universidad.

15.4 Ver el navegador en acción (en vez de invisible)

Por defecto Chrome corre invisible ("headless"). Si quieres verlo abrirse y hacer clic en los filtros en vivo (muy útil para diagnosticar si el portal del SNIES cambió de diseño), pon esta variable antes de correr:

```
$env:SNIES_HEADLESS = "0"  
python scripts/run_snies.py
```

15.5 Regenerar el dashboard localmente

```
python docs/generar_dashboard.py
```

Esto lee lo que ya esté en `Programas/` y `data/novedades/` (tal como estén en tu copia local del repositorio) y regenera todos los `.html` dentro de `docs/`. Puedes después abrir `docs/index.html` directamente con tu navegador (doble clic) para revisar los cambios de diseño antes de subirlos.

16. Problemas comunes y cómo resolverlos

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
El workflow falla en el paso "Ejecutar pipeline" con un <code>TimeoutError</code> sobre la descarga	El portal del SNIES cambió de diseño (cambió el texto de algún filtro o del botón de descarga), o está caído/muy lento	Revisar las capturas <code>debug_snies.png</code> / <code>debug_post_filtros.png</code> (se pueden descargar como "artifact" desde la corrida fallida en la pestaña Actions) para ver qué mostraba la página; puede que haya que actualizar los textos en <code>XPATHS</code> o en las llamadas a <code>_pf_select_radio</code> en <code>run_snies.py</code>
El correo nunca llega, pero el dashboard sí se actualiza	Falló el login SMTP (contraseña vencida, se usó la contraseña normal en vez de la de aplicación, la cuenta bloqueó el acceso)	Revisar el log del paso "Ejecutar pipeline", buscar <code>Error enviando el correo</code> ; regenerar la contraseña de aplicación (sección 13.1)
El dashboard no se actualiza, pero sí llega el correo	Falta el permiso "Read and write permissions" en <code>Settings</code> → <code>Actions</code> → <code>General</code> → <code>Workflow permissions</code>	Activar esa opción (sección 10.4)
El pipeline aborta diciendo "demasiados programas — probable descarga sin filtros"	El robot descargó el Excel sin que los filtros hayan surtido efecto (por lentitud del portal, por ejemplo)	Revisar <code>debug_post_filtros.png</code> ; puede ser un problema puntual que se resuelve solo reintentando (correr el workflow manualmente de nuevo)
Aparecen muchísimos "Modificados" de golpe, que no parecen reales	El snapshot anterior contra el que se comparó estaba corrupto o venía de una descarga sin filtrar de hace tiempo	Revisar cuál fue el "snapshot anterior" usado (sale en el log: <code>Snapshot ANTERIOR: Programas DD-MM-YY.xlsx</code>) y comparar manualmente ese archivo
Un programa que sé que cambió no aparece como "Modificado"	El cambio fue en un campo que no está en <code>COLS_VIGILAR</code> (ver sección 6.5)	Es el comportamiento esperado del sistema, no un error — si se quiere vigilar ese campo, hay que agregarlo a <code>COLS_VIGILAR</code>
No tengo permisos para ver/editar los Secrets en GitHub	Los secrets solo los puede administrar quien tenga rol de administrador (o "write", según configuración) en el repositorio	Pedirle a quien administre el repositorio en GitHub que te dé acceso, o que haga el cambio por ti

17. Cosas importantes que hay que saber (y detalles desactualizados)

Al revisar el proyecto a fondo se encontraron algunos puntos donde el código o los comentarios no están 100% alineados con la realidad actual — vale la pena tenerlos presentes:

1. El `README.md` dice que corre "cada día hábil", pero el cron actual (`snies_daily.yml`) solo corre **una vez por semana, los martes**. La frecuencia cambió varias veces a lo largo del proyecto (empezó siendo diaria de lunes a viernes, después semanal los lunes, y desde el 26 de junio de 2026 es semanal los martes) y el README quedó desactualizado. Si se vuelve a cambiar la frecuencia, conviene actualizar también esa frase del README.
 2. El comentario en el código dice "Configuración SMTP Office 365", pero el host configurado es `smtp.gmail.com` (Gmail), no Office 365. Es un comentario que quedó de una versión anterior del proyecto (antes sí se usaba Office 365, según el historial de commits) — el comentario no se actualizó cuando se migró a Gmail.
 3. El `README.md` menciona un archivo `scripts/run_snies_posgrado.py` ("Pipeline posgrado") que **no existe** en el repositorio actual. Por ahora el sistema **solo monitorea programas de pregrado** — el posgrado parece haber sido un plan a futuro que no se llegó a implementar (o que se quitó). Si nunca se va a implementar, valdría la pena quitar esa línea del README para no confundir a alguien nuevo.
 4. El archivo `data/Programas_pregrado_anterior.xlsx` existe en el repositorio pero **no lo usa ningún script** — parece un archivo de una versión anterior del pipeline que quedó abandonado. No afecta el funcionamiento actual, pero se podría borrar con tranquilidad si se quiere ordenar el repositorio (después de confirmar que no se necesita para nada).
 5. La carpeta `Programas/` es el **corazón real del historial**. Aunque a simple vista parece solo una bitácora de respaldo, es de ahí de donde sale tanto la comparación semana a semana como todos los gráficos de evolución histórica del dashboard. Conviene no borrar ni mover archivos de ahí manualmente.
 6. Las tres tablas de `data/novedades/` **no son "solo la última corrida"** — son el acumulado de *toda* la historia del monitor. Cuando se quiera saber "¿qué pasó la última semana?", hay que fijarse en la columna `FECHA_OBTENCION` (o usar el filtro por fecha del dashboard), no asumir que el archivo entero es de la corrida más reciente.
-

18. Glosario de términos técnicos

- **SNIES:** Sistema Nacional de Información de la Educación Superior — la base de datos pública del Ministerio de Educación de Colombia con todos los programas académicos del país.
- **Snapshot:** una "foto" del estado de los datos en un momento específico. Cada Excel en `Programas/` es un snapshot.
- **Scraping / Web scraping:** técnica de extraer información de una página web automatizando un navegador, cuando no existe una forma más directa (API) de obtener esos datos.
- **Selenium:** la librería de Python que controla un navegador real (Chrome, en este caso) de forma automática.
- **Headless:** modo en el que un navegador corre sin mostrar ninguna ventana en pantalla — funciona igual por dentro, pero no se ve.
- **XPath:** una forma estándar de "darle direcciones" a un elemento dentro de una página web (ej: "el botón que tiene un texto que dice tal cosa"), para que un programa lo pueda encontrar y hacer clic en él.
- **AJAX:** técnica con la que una página web le pide datos nuevos al servidor y actualiza solo una parte de la pantalla, sin recargar la página completa.
- **JSF / PrimeFaces:** una tecnología de programación web (común en sistemas del gobierno colombiano) que genera identificadores técnicos que cambian con cada actualización del sistema — por eso el robot evita depender de ellos y usa el texto visible.
- **SMTP:** el protocolo estándar de internet para enviar correos electrónicos por programación.
- **App Password / Contraseña de aplicación:** una clave especial que generan Gmail (y otros proveedores) para que un programa pueda enviar correos sin usar la contraseña normal de la cuenta, por seguridad.
- **GitHub Actions:** el servicio de GitHub para correr tareas automáticas (workflows) en la nube, programadas o manuales.
- **Workflow:** un archivo de configuración (`.yaml`) que le dice a GitHub Actions exactamente qué pasos ejecutar.
- **Cron:** la notación estándar para expresar "a qué hora y qué días" debe correr algo de forma automática.
- **Secret:** un valor sensible (contraseña, token) guardado de forma cifrada en GitHub, que un workflow puede usar sin exponerlo en el código.
- **GitHub Pages:** el servicio gratuito de GitHub para publicar páginas web estáticas directamente desde un repositorio.
- **CINE (Clasificación Internacional Normalizada de la Educación):** un sistema de clasificación de áreas del conocimiento definido por la UNESCO, usado internacionalmente para poder comparar programas académicos de distintos países/sistemas.
- **Núcleo Básico del Conocimiento (NBC):** otra clasificación de áreas de conocimiento, propia del sistema educativo colombiano (distinta del CINE, aunque con propósito similar).
- **Dashboard:** panel visual con gráficos, indicadores y tablas para explorar datos de forma interactiva — en este proyecto es la página web publicada en GitHub Pages.
- **Plotly:** la librería de JavaScript que dibuja los gráficos interactivos del dashboard (permite hacer zoom, ver valores exactos al pasar el mouse, etc.).
- **JSON:** un formato de texto muy usado para representar datos estructurados (algo parecido a un diccionario de Python) — es el formato en el que el dashboard recibe todos sus datos (`docs/dashboard_data.json`).

Manual generado a partir de una revisión completa del código fuente del repositorio `snies-monitor` en julio de 2026. Si el código cambia de forma importante en el futuro, conviene revisar que este manual siga reflejando la realidad, especialmente las secciones 4, 6, 10 y 17.